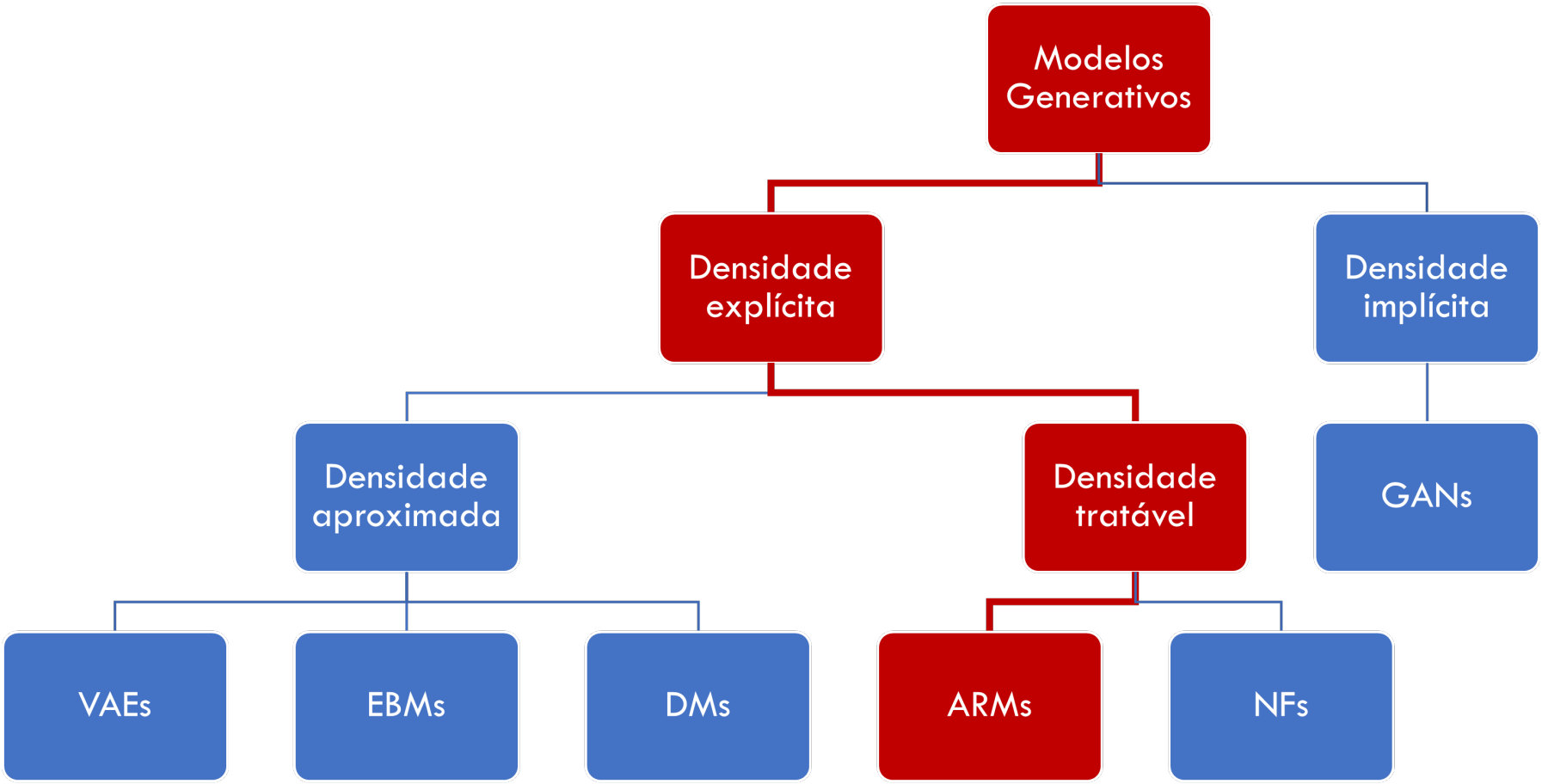
Modelos Auto-regressivos - parte 1

Inteligência Artificial Generativa

Modelos auto-regressivos [*autoregressive models*, ARMs]



Modelos auto-regressivos [*autoregressive models*, ARMs]

- Modelos auto-regressivos fatoram a distribuição conjunta dos dados $\boldsymbol{x} = [x_1, x_2, \ldots, x_D]$ como um produto de distribuições condicionais:

$$p(\boldsymbol{x}) = \prod_{d=1}^{D} p(x_d \mid \underbrace{x_1, \ldots, x_{d-1}}_{\boldsymbol{x}_{<d}}) \quad \text{(regra da cadeia da probabilidade)}$$

$$= p(x_1) \prod_{d=2}^{D} p(x_d \mid \boldsymbol{x}_{<d}) \quad \text{(notação compacta)}$$

- Amostragem sequencial:
  1. sortear $x_1 \sim p(x_1)$;
  2. para $d \in \{2, \ldots, D\}$: sortear $x_d \sim p(x_d \mid \boldsymbol{x}_{<d})$.
- Aplicações: geração de texto, imagens, áudio, etc.

Como modelar as distribuições condicionais nos ARMs?

Podemos utilizar, por exemplo:

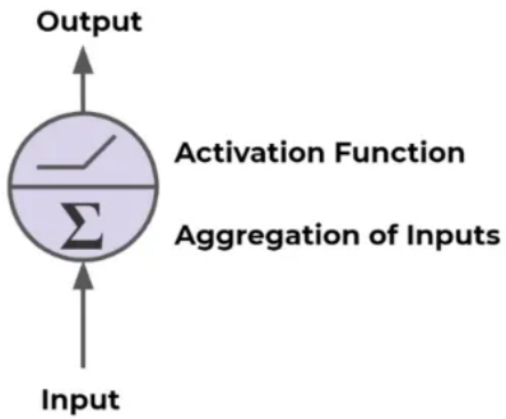
- **Redes neurais recorrentes**
- Transformadores [*transformers*]
- Redes neurais convolucionais

O que são Redes Neurais Recorrentes (RNNs)?

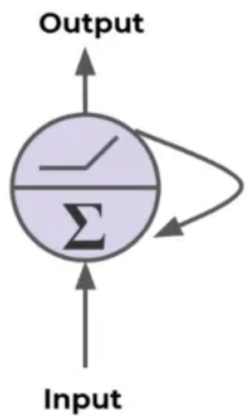
- Prever o futuro é uma atividade humana recorrente: completar frases, antecipar cheiros ou prever ações.
- Redes neurais recorrentes [*recurrent neural networks*, RNNs] são modelos projetados para lidar com dados sequenciais e predizer valores futuros a partir de padrões observados (assumindo que tais padrões permaneçam válidos).
- RNNs operam sobre **sequências de comprimento arbitrário**, ao contrário de redes de propagação direta [*feedforward*], que exigem entradas de tamanho fixo.
- RNNs são adequadas para diferentes modalidades de dados sequenciais:
  - **Séries temporais** (temperatura, tráfego, consumo energético, etc.)
  - **Texto**
  - **Áudio**

Neurônio recorrente

- Neurônio de propagação direta [*feedforward*]
  - Fluxo unidirecional: entrada $\rightarrow$ saída.
- Neurônio recorrente
  - Em cada instante de tempo t, o neurônio recebe:
    1. a entrada atual $\boldsymbol{x}_{(t)}$;
    2. sua própria saída (escalar) anterior $\hat{y}_{(t-1)}$. Assume-se: $\hat{y}_{(-1)} = 0$.



Neurônio *feedforward*



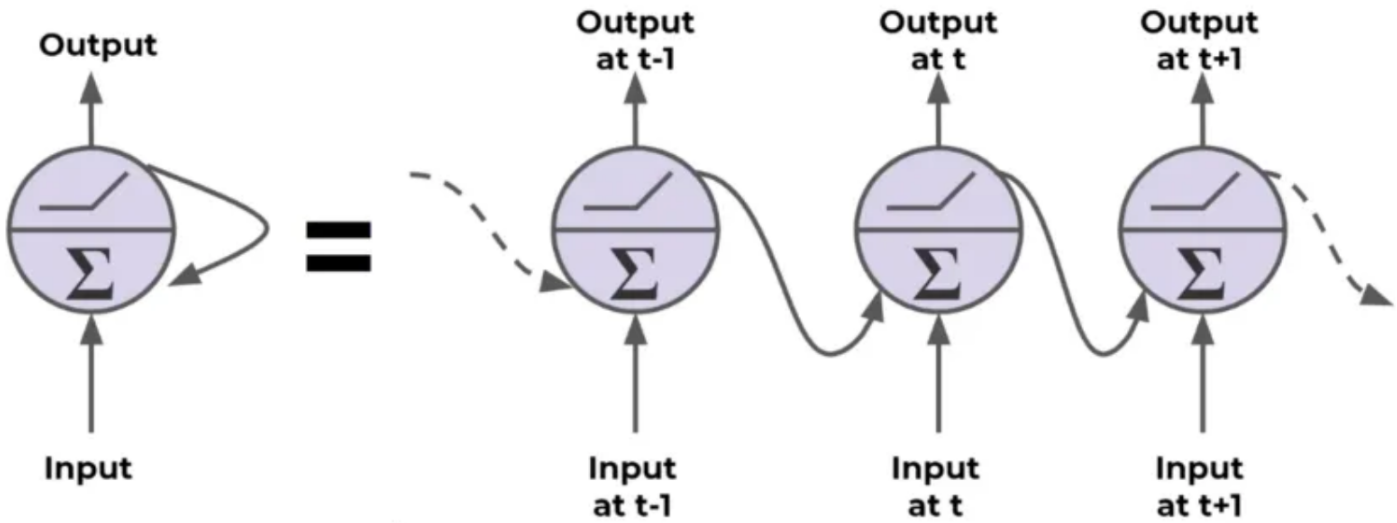
Neurônio recorrente

---

https://becominghuman.ai/a-to-z-about-recurrent-neural-network-rnn-43562f7c49c4

Desdobramento no Tempo [*unrolling through time*]

- O neurônio recorrente pode ser representado **desdobrado no tempo** [*unrolled through time*], **repetindo-se** conceitualmente a cada instante de tempo t.
- Todas as cópias compartilham os **mesmos pesos** e a **mesma função de ativação**.

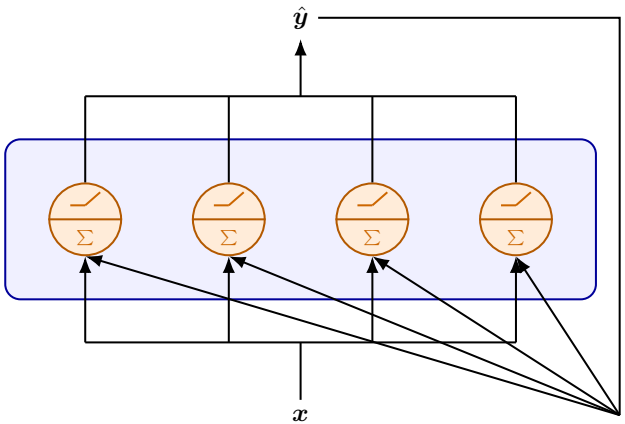


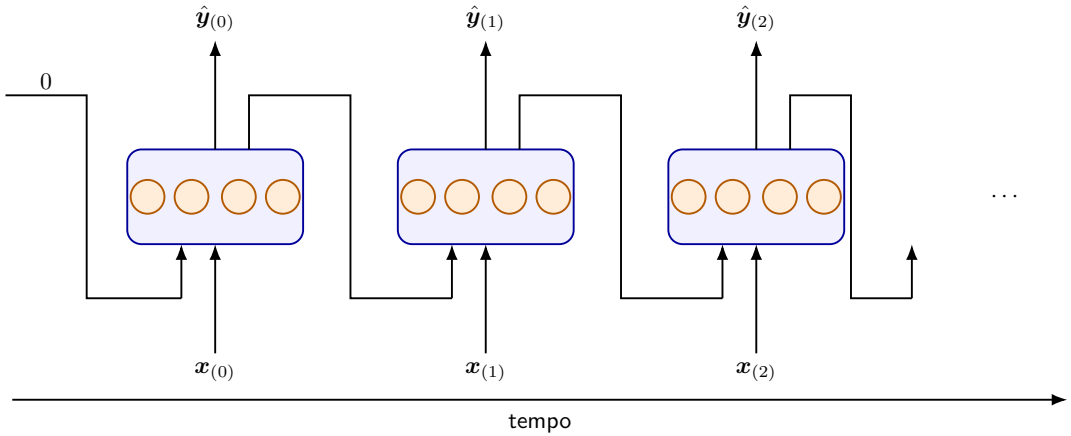
---

https://becominghuman.ai/a-to-z-about-recurrent-neural-network-rnn-43562f7c49c4

Camada recorrente

- Em uma camada RNN, no instante t:
  - entrada: vetor $\boldsymbol{x}_{(t)}$;
  - vetor de saída no instante de tempo anterior: vetor $\hat{\boldsymbol{y}}_{(t-1)}$.





Camada recorrente

Saída de uma camada recorrente para uma única instância

$$\hat{\boldsymbol{y}}_{(t)} = \phi\left(\boldsymbol{W}_x^\top \boldsymbol{x}_{(t)} + \boldsymbol{W}_{\hat{y}}^\top \hat{\boldsymbol{y}}_{(t-1)} + \boldsymbol{b}\right)$$

A saída da camada recorrente no instante t depende de:

- entrada atual $\boldsymbol{x}_{(t)}$
- saída do instante de tempo anterior $\hat{\boldsymbol{y}}_{(t-1)}$
- matriz de pesos associados à $\boldsymbol{x}_{(t)}$: $\boldsymbol{W}_x$
- matriz de pesos associados à $\hat{\boldsymbol{y}}_{(t-1)}$: $\boldsymbol{W}_{\hat{y}}$
- vetor de viés [*bias*]: $\boldsymbol{b}$
- função de ativação $\phi(\cdot)$ (ex.: tangente hiperbólica, ReLU)

Camada recorrente

Saída de uma camada recorrente para um mini-lote

$$\hat{\boldsymbol{Y}}_{(t)} = \phi\left(\boldsymbol{X}_{(t)}\boldsymbol{W}_x + \hat{\boldsymbol{Y}}_{(t-1)}\boldsymbol{W}_{\hat{y}} + \boldsymbol{b}\right)$$

$$= \phi\left(\begin{bmatrix}\boldsymbol{X}_{(t)} & \hat{\boldsymbol{Y}}_{(t-1)}\end{bmatrix}\boldsymbol{W} + \boldsymbol{b}\right), \quad \text{com} \quad \boldsymbol{W} = \begin{bmatrix}\boldsymbol{W}_x \\ \boldsymbol{W}_{\hat{y}}\end{bmatrix}$$

Seja m o tamanho do mini-lote, d o número de atributos da entrada e n o número de neurônios da camada recorrente, então:

- $\boldsymbol{X}_{(t)}$: matriz $m \times d$ contendo m entradas no passo de tempo t.
- $\hat{\boldsymbol{Y}}_{(t)}$: matriz $m \times n$ contendo m saídas da camada recorrente no passo de tempo t.
- $\boldsymbol{W}_x$: matriz $d \times n$ contendo os pesos associados às entradas no passo de tempo t.
- $\boldsymbol{W}_{\hat{y}}$: matriz $n \times n$ contendo os pesos associados às saídas no passo de tempo $t-1$.
- $\boldsymbol{b}$: vetor contendo os vieses de cada um dos n neurônios da camada recorrente.
- $\phi(\cdot)$: função de ativação.
- $\boldsymbol{W}$: matriz $(d+n) \times n$ contendo a concatenação vertical de $\boldsymbol{W}_x$ e $\boldsymbol{W}_{\hat{y}}$.
- $\begin{bmatrix}\boldsymbol{X}_{(t)} & \hat{\boldsymbol{Y}}_{(t-1)}\end{bmatrix}$: matriz $m \times (d+n)$ contendo a concatenação horizontal de $\boldsymbol{X}_{(t)}$ e $\hat{\boldsymbol{Y}}_{(t-1)}$.

Células de Memória em RNNs

- Neurônios recorrentes possuem **memória**: suas saídas em qualquer instante de tempo dependem das entradas oriundas de todos os instantes de tempo anteriores.
- Uma estrutura capaz de preservar algum estado ao longo do tempo denomina-se **célula de memória**.
- Um neurônio recorrente (ou uma camada recorrente) constitui uma **célula básica**.

Observação

O estado oculto de uma célula e sua saída podem ser diferentes.

- $x_{(t)}$: entrada no tempo t.
- $h_{(t)}$: estado oculto de uma célula no tempo t (h de *hidden* (oculto))

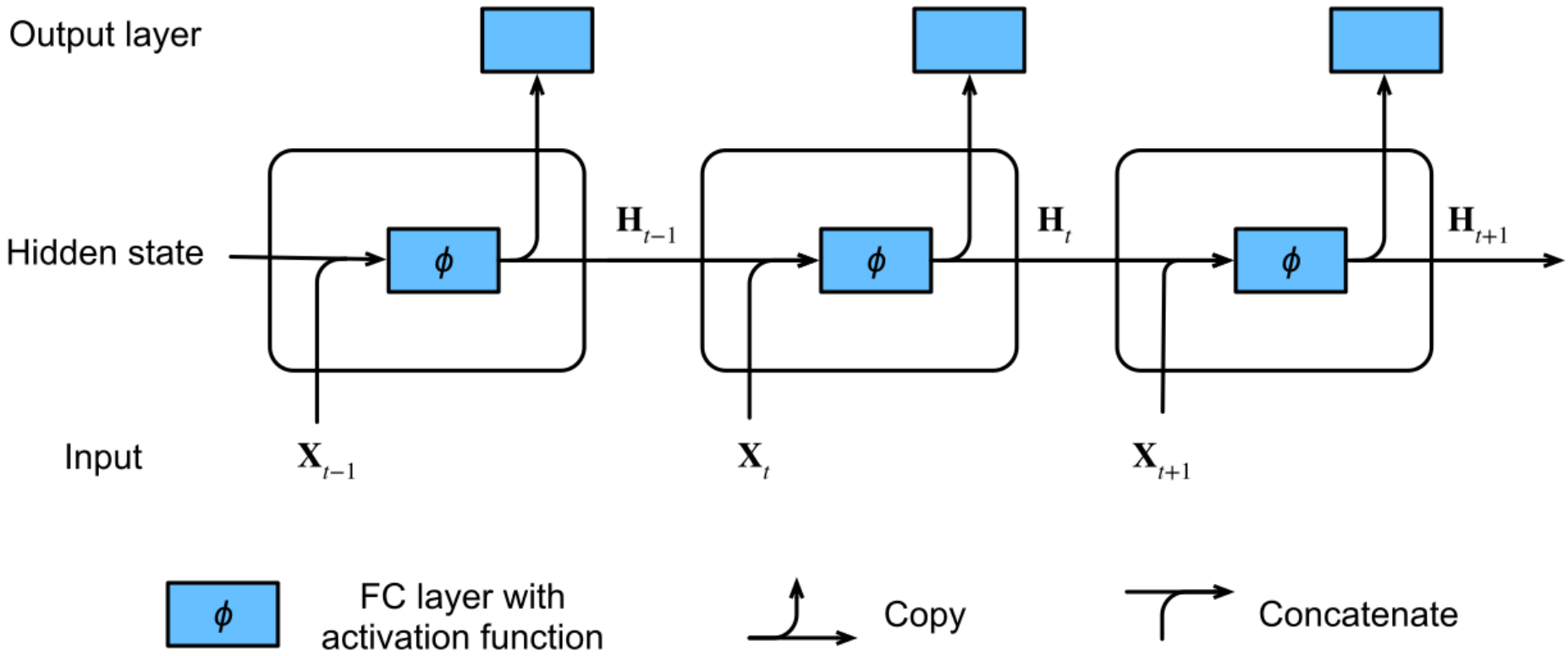
$$h_{(t)} = f(x_{(t)}, h_{(t-1)})$$

- $\hat{y}_{(t)}$: saída no tempo t.

$$\hat{y}_{(t)} = g(h_{(t)})$$

  - Em células básicas: $\boxed{\hat{y}_{(t)} = h_{(t)}}$.

Arquitetura RNN Básica

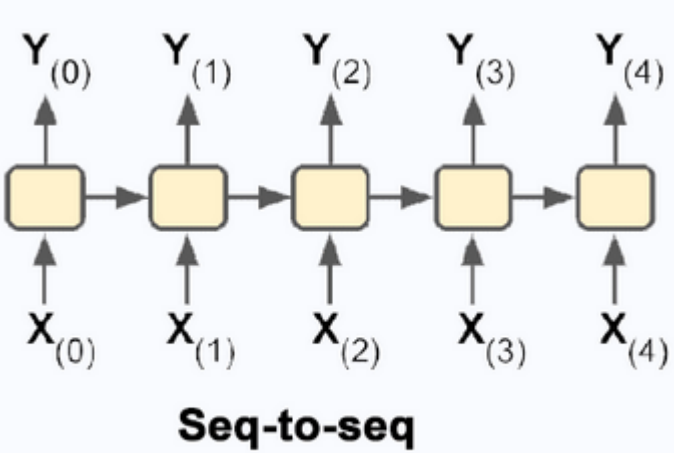


A. Zhang et al., *Dive into Deep Learning*, Cambridge University Press, 2023.

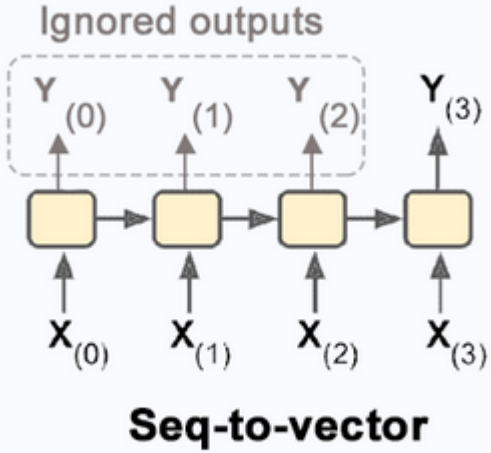
Arquiteturas de RNNs

Arquitetura	**Entrada**	**Saída**	**Exemplo**
Sequência à sequência [*sequence-to-sequence*]	Uma sequência	Uma sequência	Predição de séries temporais
Sequência à vetor [*sequence-to-vector*]	Uma sequência	Um vetor (a última saída).	Análise de sentimentos.
Vetor à sequência [*vector-to-sequence*]	Mesmo vetor repetido a cada passo de tempo	Uma sequência de vetores	Geração de legendas para imagens.

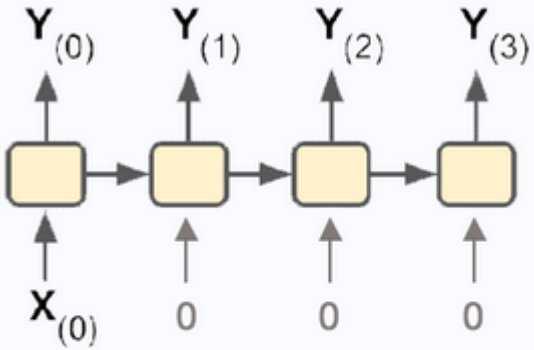
Arquiteturas de RNNs



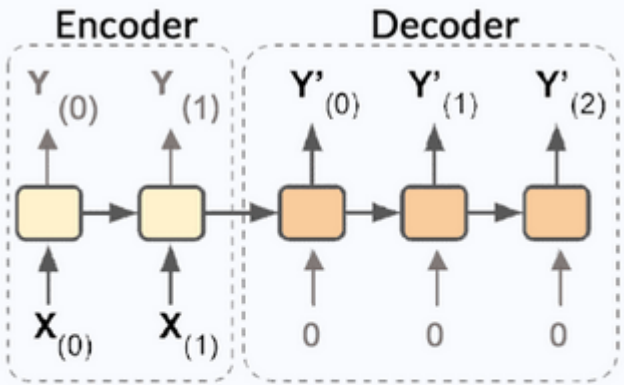
Seq-to-seq



Seq-to-vector



Vector-to-seq



Encoder-Decorder

https://k21academy.com/ai-ml/recurrent-neural-networks/

Arquiteturas de RNNs

- Arquitetura **encoder-decoder**
  - Em tradução automática, o final da frase original pode influenciar o início da frase traduzida.
  - É necessário observar a frase inteira antes de gerar a tradução.
  - Estruturas diretas sequência-para-sequência falham nesse aspecto.
  - Arquitetura **encoder-decoder** combina duas redes recorrentes:
    - **Encoder:** rede neural sequência à vetor
    - **Decoder:** rede neural vetor à sequência

Treinamento de RNNs

- O treinamento de RNNs é realizado utilizando o algoritmo da retropropagação através do tempo [***Backpropagation Through Time*, BPTT**]
  1. A RNN é **desdobrada no tempo** [*unrolling*].
  2. Aplica-se o algoritmo da retropropagação [*backpropagation*] na rede desdobrada.

BPTT: Passo Forward em RNNs

- A passagem direta percorre todos os instantes temporais da rede desdobrada.
- A função de perda é então calculada como:

$$\mathcal{L}(\boldsymbol{Y}_{(0)}, \ldots, \boldsymbol{Y}_{(T)}, \hat{\boldsymbol{Y}}_{(0)}, \ldots, \hat{\boldsymbol{Y}}_{(T)}) = \sum_{t=0}^{T} \ell_{(t)} \left(\boldsymbol{Y}_{(t)}, \hat{\boldsymbol{Y}}_{(t)} \right)$$

  Onde
  - $\boldsymbol{Y}_{(i)}$: i-ésimo alvo
  - $\hat{\boldsymbol{Y}}_{(i)}$: i-ésima predição
  - T: último instante de tempo
- A função de perda incorpora um subconjunto de saídas adequado á tarefa desempenhada (e a arquitetura). Por exemplo, em modelos sequência à vetor, a função de perda inclui somente a saída $\hat{\boldsymbol{Y}}_{(T)}$.

BPTT: Passo Backward em RNNs

- Saídas ignoradas na função de perda não propagam gradientes.
- Os gradientes das saídas consideradas são propagados através das cópias da rede neural ao longo do tempo.
- Parâmetros compartilhados recebem gradientes acumulados de vários instantes de tempo.

BPTT: Atualização dos Parâmetros

- Após o cálculo dos gradientes no passo backward, realiza-se a atualização dos parâmetros utilizando um algoritmo de otimização de forma análoga ao algoritmo de retropropagação padrão em redes *feedforward*.

RNNs em Sequências Longas

Desafios ao lidar com sequências longas

Ao desenrolar uma RNN por muitos passos de tempo, surgem 2 problemas:

1. **Gradientes instáveis**:
   - a RNN desenrolada vira uma rede profunda no tempo;
   - os gradientes podem explodir ou desvanecer.
2. **Memória de curto prazo limitada**:
   - a RNN esquece gradualmente as primeiras entradas da sequência.

RNNs em Sequências Longas

O problema dos gradientes instáveis em RNNs

Por que RNNs sofrem com gradientes instáveis?

- Ao desenrolar uma RNN por muitos passos:
  - cada passo adiciona mais uma "camada" à rede no tempo;
  - isso cria uma rede **profunda** com pesos compartilhados no tempo.
- Efeitos conhecidos de redes profundas reaparecem:
  - gradientes podem ficar muito pequenos (treinamento lento);
  - gradientes podem explodir (treinamento instável).
- Em RNNs, há ainda um efeito acumulativo:
  - qualquer alteração nos pesos afeta **todos** os passos;

Estratégias herdadas do treinamento de redes profundas

- Várias técnicas que mitigam o problema dos gradientes instáveis em redes profundas *feedforward* podem também ser aplicadas em RNNs:
  - Boa inicialização dos parâmetros.
  - Otimizadores mais rápidos.
  - Utilizar dropout
- Mas nem todas as técnicas se transferem bem:
  - Normalização em lote [*batch normalization*] nas RNNs costuma trazer pouco ganho de desempenho em RNNs e ainda tornar o treinamento mais lento.
  - Funções de ativação não saturantes (ex.: ReLU) **podem até piorar** a instabilidade em RNNs.

Intuição: por que a ReLU pode levar a explosão?

- Se uma atualização dos pesos aumentar ligeiramente as saídas no primeiro passo de tempo, então as demais saídas podem aumentar até **explodirem**.

Por quê?

Intuição: por que ReLU pode levar a explosão?

- Se uma atualização dos pesos aumentar ligeiramente as saídas no primeiro passo de tempo, então as demais saídas podem aumentar até **explodirem**.
  - Porque nas RNNs, os pesos são compartilhados em todos os passos de tempo t.
  - Uma função não saturante como `ReLU` não impõe um limite superior às saídas dos neurônios. Logo, não impede esse crescimento descontrolado.
- De maneira similar, os gradientes podem também explodir.

Táticas para estabilizar o treinamento

- Reduzir a taxa de aprendizagem;
- Utilizar ativações saturantes (ex.: $\tanh$);
- Monitorar o tamanho dos gradientes;
- Limitar o tamanho dos gradientes [*gradient clipping*];
- Utilizar normalização por camada [*layer normalization*]:
  - Normaliza ao longo dos atributos [*features*] de cada instância, em vez de normalizar ao longo do lote [*batch*].

RNNs em Sequências Longas

O problema da memória de curto prazo limitada em RNNs

RNNs e o Esquecimento da Informação Inicial

Problema central

A cada passo de tempo, a RNN transforma seu estado interno e parte da informação é perdida.

- O estado da RNN acumula sucessivas transformações ao longo da sequência.
- Informações das primeiras entradas vão se tornando gradualmente irrelevantes.
- Após muitos passos, o estado pode não conter quase nada do início da sequência.
- Para tarefas como traduzir frases longas, isso pode ser um verdadeiro obstáculo.

Por que Precisamos de Memória de Longo Prazo?

Situação típica

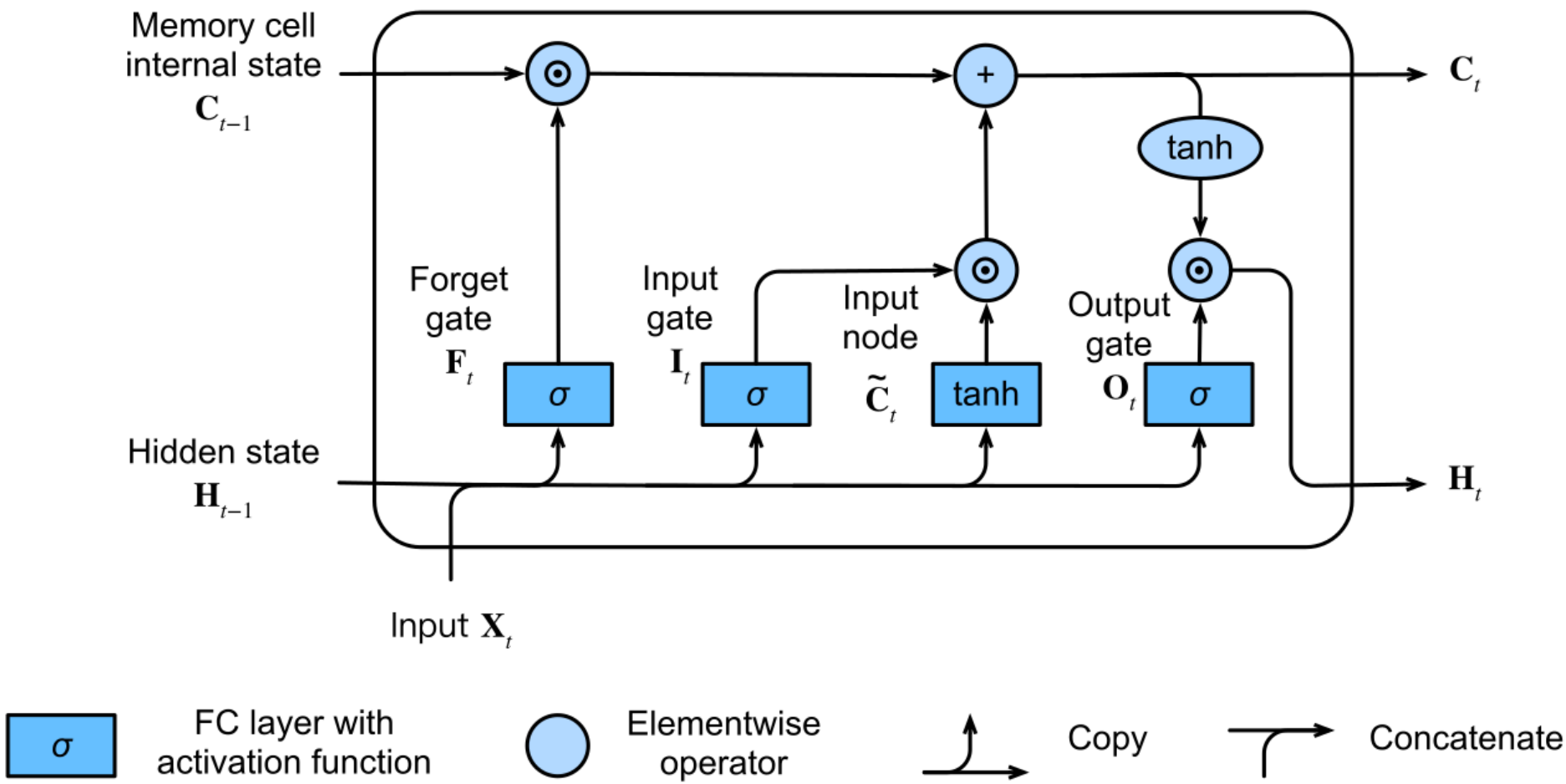
A rede precisa "lembrar" algo dito no começo da sequência quando chega ao final (por exemplo, em tradução automática).

- RNNs simples não conseguem preservar bem informações distantes no tempo.
- Isso limita a capacidade de modelar dependências de longo alcance.
- Para contornar esse problema, foram propostas células com **memória de longo prazo**.
- Essas células são tão eficazes que as células básicas quase não são mais usadas.

Memória de curto prazo longa [Long Short-Term Memory, LSTM]

- Quando usada como uma "caixa-preta":
  - ocupa o papel de uma célula de RNN tradicional;
  - desempenho significativamente melhor;
  - o treinamento tende a convergir mais rápido;
  - captura padrões de mais longo prazo nos dados.

Arquitetura LSTM



A. Zhang et al., *Dive into Deep Learning*, Cambridge University Press, 2023.

Arquitetura LSTM

Estados internos

A LSTM mantém 2 vetores de estado distintos em cada passo de tempo.

1. $h_{(t)}$
   - estado de curto prazo
   - "h" de "hidden" (oculto)
   - é também saída da célula $\hat{y}_{(t)}$ no tempo t.
   - $h_{(t)}$ reflete o que é relevante "agora".
2. $c_{(t)}$
   - estado de longo prazo
   - "c" de "cell" (célula)
   - $c_{(t)}$ acumula memórias que podem atravessar muitos passos de tempo.

Como a célula decide o que manter, apagar ou expor?

Quatro Camadas Internas na LSTM

- A cada passo de tempo, são combinados:
  - a entrada atual $x_{(t)}$
  - o estado de curto prazo anterior $h_{(t-1)}$
- Estes dois vetores alimentam **quatro camadas totalmente conectadas**:
  1. Uma camada principal produz o sinal $\tilde{c}_{(t)}$.
     - Nas RNNs tradicionais, existe apenas essa camada, a qual produz o análogo de $\tilde{c}_{(t)}$ como estado $h_{(t)}$ e saída $\hat{y}_{(t)}$.
     - Na LSTM, o conteúdo de $\tilde{c}_{(t)}$ é filtrado e parcialmente armazenado em $c_{(t)}$.
  2. Três camadas adicionais produzem sinais de controle (portas).

Portas da LSTM

Função de ativação das portas

As portas usam a função logística, produzindo valores entre 0 e 1.

- Cada porta controla uma operação de multiplicação elemento a elemento:
  - valores próximos de 0 $\Rightarrow$ "fecham" a passagem,
  - valores próximos de 1 $\Rightarrow$ "abrem" a passagem.

Porta	**Função**
de esquecimento ($f_{(t)}$)	Decide o que apagar do estado de longo prazo.
de entrada ($i_{(t)}$)	Decide que partes de $\tilde{c}_{(t)}$ serão incorporadas a $c_{(t)}$.
de saída ($o_{(t)}$)	Decide que partes de $c_{(t)}$ serão expostas em $h_{(t)}$ e $\hat{y}_{(t)}$.

Fluxo dos estados de longo e de curto prazo na LSTM

1. $c_{(t-1)}$ entra na célula no tempo t.
2. sob o controle da **porta de esquecimento** [forget gate], é decidido o que será preservado ou atenuado em $c_{(t-1)}$.
3. sob o controle da **porta de entrada** [input gate], novas informações são adicionadas a $c_{(t-1)}$ já filtrado a filtragem pela porta de esquecimento, gerando $c_{(t)}$.
4. $h_{(t)}$ é obtido pela transformação de $c_{(t)}$ por `tanh` e subsequente filtragem pela **porta de saída** [output gate].
   - A saída da célula no tempo t é $\hat{y}_{(t)} = h_{(t)}$.

O Que uma LSTM Pode Aprender a Fazer?

- Reconhecer entradas importantes em momentos específicos da sequência (porta de entrada).
- Armazenar essas informações no estado de longo prazo $c_{(t)}$.
- Preservar essas memórias enquanto forem relevantes (porta de esquecimento).
- Extrair essas memórias quando necessário (porta de saída).
- Essa capacidade torna a LSTM eficaz em:
  - séries temporais;
  - textos extensos;
  - gravações de áudio;
  - entre outros.

Equações da LSTM para uma única instância $\boldsymbol{x}_{(t)}$

$$
\begin{aligned}
i_{(t)} &= \sigma(W_{xi}^{\top} x_{(t)} + W_{hi}^{\top} h_{(t-1)} + b_i) \\
f_{(t)} &= \sigma(W_{xf}^{\top} x_{(t)} + W_{hf}^{\top} h_{(t-1)} + b_f) \\
o_{(t)} &= \sigma(W_{xo}^{\top} x_{(t)} + W_{ho}^{\top} h_{(t-1)} + b_o) \\
\tilde{c}_{(t)} &= \tanh\left(W_{xg}^{\top} x_{(t)} + W_{hg}^{\top} h_{(t-1)} + b_g\right) \\
c_{(t)} &= f_{(t)} \odot c_{(t-1)} + i_{(t)} \odot \tilde{c}_{(t)} \\
\hat{y}_{(t)} &= h_{(t)} = o_{(t)} \odot \tanh\left(c_{(t)}\right)
\end{aligned}
$$

Parâmetros da LSTM

- Matrizes de pesos associadas à entrada $\boldsymbol{x}_{(t)}$:

$$\boldsymbol{W}_{xi},\ \boldsymbol{W}_{xf},\ \boldsymbol{W}_{xo},\ \boldsymbol{W}_{xg}$$

- Matrizes de pesos associadas ao estado de curto prazo anterior $\boldsymbol{h}_{(t-1)}$:

$$\boldsymbol{W}_{hi},\ \boldsymbol{W}_{hf},\ \boldsymbol{W}_{ho},\ \boldsymbol{W}_{hg}$$

- Vetores de viés:

$$\boldsymbol{b}_{i},\ \boldsymbol{b}_{f},\ \boldsymbol{b}_{o},\ \boldsymbol{b}_{g}$$

Variantes da LSTM

- Várias variantes da célula LSTM foram propostas na literatura.
- Uma variante particularmente popular é a **GRU (Gated Recurrent Unit)**.

Unidade Recorrente com Portas [Gated Recurrent Unit, GRU]

- A célula GRU consiste em uma versão simplificada da célula LSTM.
- Apesar da simplificação, costuma apresentar desempenho comparável ao da LSTM.

GRU como Simplificação da LSTM

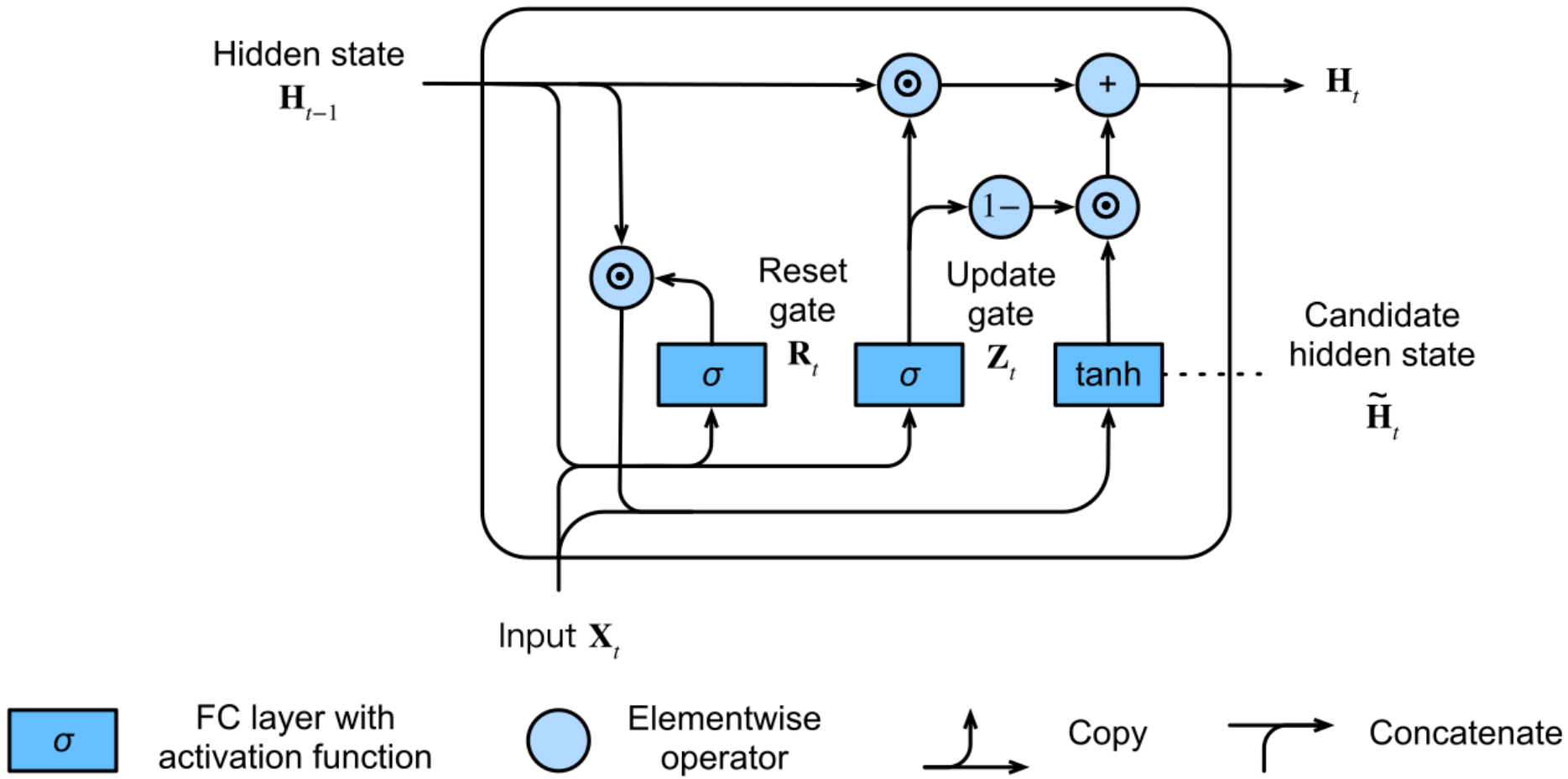
Ideia central

Manter os benefícios da memória de longo prazo, reduzindo o número de estados e de portas explícitas.

- Na LSTM, existem 2 estados distintos:
  - estado de curto prazo $\boldsymbol{h}_{(t)}$;
  - estado de longo prazo $\boldsymbol{c}_{(t)}$.
- Na GRU, esses dois vetores são **fundidos em um único estado**:

$$\boldsymbol{h}_{(t)} \quad \text{(único vetor de estado)}$$

Arquitetura GRU



A. Zhang et al., *Dive into Deep Learning*, Cambridge University Press, 2023.

Controle Conjunto de Esquecimento e Escrita

- Na LSTM, a porta de esquecimento [*forget gate*] e a porta de entrada [*input gate*] são controladas separadamente.
- Na GRU, um **único controlador** $\boldsymbol{z}_{(t)}$ assume esse papel combinado de "esquecer" e "escrever".
  - Se $\boldsymbol{z}_{(t)}$ "abre" a porta de esquecimento [forget gate], a escrita [input gate] é "fechada".
  - Se $\boldsymbol{z}_{(t)}$ "fecha" a porta de esquecimento [forget gate], a escrita [input gate] é "aberta".
  - Lógica "apaga depois escreve": sempre que uma nova memória é armazenada, a "posição" correspondente é apagada antes.

Saída da GRU e Novo Controlador

- Diferentemente da LSTM, a GRU **não possui porta de saída** explícita.
- O vetor de estado $\boldsymbol{h}_{(t)}$ é totalmente exposto a cada passo de tempo.
- Surge um novo controlador $\boldsymbol{r}_{(t)}$ com função distinta:
  - $\boldsymbol{r}_{(t)}$ decide **que parte do estado anterior** $\boldsymbol{h}_{(t-1)}$ será mostrada à camada principal que produz $\tilde{\boldsymbol{h}}_{(t)}$.
- Assim, a combinação $(\boldsymbol{z}_{(t)}, \boldsymbol{r}_{(t)})$ determina:
  - quanto do estado anterior é preservado;
  - quanto do estado será usado para calcular o novo conteúdo.

Equações da GRU para uma única instância $\boldsymbol{x}_{(t)}$

$$
\begin{aligned}
\boldsymbol{z}_{(t)} &= \sigma(\boldsymbol{W}_{xz}^{\top}\boldsymbol{x}_{(t)} + \boldsymbol{W}_{hz}^{\top}\boldsymbol{h}_{(t-1)} + \boldsymbol{b}_z) \\
\boldsymbol{r}_{(t)} &= \sigma(\boldsymbol{W}_{xr}^{\top}\boldsymbol{x}_{(t)} + \boldsymbol{W}_{hr}^{\top}\boldsymbol{h}_{(t-1)} + \boldsymbol{b}_r) \\
\tilde{\boldsymbol{h}}_{(t)} &= \tanh\big(\boldsymbol{W}_{xg}^{\top}\boldsymbol{x}_{(t)} + \boldsymbol{W}_{hg}^{\top}(\boldsymbol{r}_{(t)} \odot \boldsymbol{h}_{(t-1)}) + \boldsymbol{b}_g\big) \\
\hat{\boldsymbol{y}}_{(t)} &= \boldsymbol{h}_{(t)} = \boldsymbol{z}_{(t)} \odot \boldsymbol{h}_{(t-1)} + \big(\mathbf{1} - \boldsymbol{z}_{(t)}\big) \odot \tilde{\boldsymbol{h}}_{(t)}
\end{aligned}
$$

Parâmetros da GRU

- Matrizes de pesos associadas à entrada $\boldsymbol{x}_{(t)}$:

$$\boldsymbol{W}_{xz},\ \boldsymbol{W}_{xr},\ \boldsymbol{W}_{xg}$$

- Matrizes de pesos associadas ao estado anterior $\boldsymbol{h}_{(t-1)}$:

$$\boldsymbol{W}_{hz},\ \boldsymbol{W}_{hr},\ \boldsymbol{W}_{hg}$$

- Vetores de viés:

$$\boldsymbol{b}_{z},\ \boldsymbol{b}_{r},\ \boldsymbol{b}_{g}$$

Interpretação das Equações da GRU

- $\boldsymbol{z}_{(t)}$ controla a **mistura** entre:
  - o estado anterior $\boldsymbol{h}_{(t-1)}$ (memória a ser preservada);
  - o novo conteúdo $\tilde{\boldsymbol{h}}_{(t)}$ (memória a ser atualizada).
- O termo $\boldsymbol{r}_{(t)} \odot \boldsymbol{h}_{(t-1)}$ determina quanto do estado anterior é **mostrado** à camada que produz $\tilde{\boldsymbol{h}}_{(t)}$.
- Quando componentes de $\boldsymbol{r}_{(t)}$ são próximos de 0, partes de $\boldsymbol{h}_{(t-1)}$ são "ignoradas" na geração de $\tilde{\boldsymbol{h}}_{(t)}$.
- Quando componentes de $\boldsymbol{z}_{(t)}$ são próximos de 1, a GRU prioriza $\boldsymbol{h}_{(t-1)}$; quando próximos de 0, prioriza $\tilde{\boldsymbol{h}}_{(t)}$.

LSTM vs GRU: Visão Comparativa

Aspecto	**LSTM**	**GRU**
Vetores de estado	$\boldsymbol{h}_{(t)}$ (curto prazo) $\boldsymbol{c}_{(t)}$ (longo prazo)	$\boldsymbol{h}_{(t)}$ (único vetor de estado)
Memória de longo prazo	Estado separado $\boldsymbol{c}_{(t)}$ [*cell state*]	Memória embutida diretamente em $\boldsymbol{h}_{(t)}$
Portas principais	$i_{(t)}$ [*input gate*] $f_{(t)}$ [*forget gate*] $o_{(t)}$ [*output gate*]	$\boldsymbol{z}_{(t)}$ (controlador que assume os papéis de [*input gate*] e [*forget gate*]) $\boldsymbol{r}_{(t)}$ (controla quanto de $\boldsymbol{h}_{(t-1)}$ é mostrado) Não há [*output gate*]
Controle de esquecimento e escrita	"esquecer"/"escrever" em $\boldsymbol{c}_{(t)}$ controlados por $f_{(t)}$ e $i_{(t)}$	"esquecer"/"escrever" em $\boldsymbol{h}_{(t)}$ controlados por $\boldsymbol{z}_{(t)}$
Saída visível	Saída filtrada por $o_{(t)}$: $\hat{\boldsymbol{y}}_{(t)} = \boldsymbol{h}_{(t)} = o_{(t)} \odot \tanh\left(\boldsymbol{c}_{(t)}\right)$	Saída igual ao estado completo exposto em cada passo: $\hat{\boldsymbol{y}}_{(t)} = \boldsymbol{h}_{(t)}$

LSTM, GRU e Limitações em Sequências Muito Longas

- Células LSTM e GRU estão entre os principais responsáveis pelo sucesso das RNNs.
- Elas conseguem lidar com sequências bem mais longas do que RNNs tradicionais.
- Ainda assim, têm memória de curto prazo **limitada**.

Avaliação de qualidade de modelos de linguagem

- Um bom modelo de linguagem deve predizer os próximos *tokens* com precisão.
- Exemplo: sejam as seguintes continuações de "Está chovendo" propostas por diferentes modelos de linguagem
  1. "Está chovendo lá fora" $\rightarrow$ modelo com a melhor predição
  2. "Está chovendo abacates e canecas" $\rightarrow$ modelo com uma predição semanticamente sem sentido
  3. "Está chovendo sjdfhosaifhd aisj@" $\rightarrow$ modelo que não está adequadamente ajustado aos dados

Avaliação de qualidade de modelos de linguagem

- Qual o problema de avaliar o modelo utilizando a **verossimilhança da sequencia**?

Avaliação de qualidade de modelos de linguagem

- Qual o problema de avaliar o modelo com a **verossimilhança da sequencia**?
  - difícil comparar modelos em textos de comprimentos diferentes: textos mais longos têm probabilidade menor.
- Queremos medir quão "surpreendente" é um texto sob um modelo. **Que quantidade podemos utilizar?**

Avaliação de qualidade de modelos de linguagem

- Podemos usar a entropia cruzada média sobre a sequência:

$$\text{entropia cruzada média} = \frac{1}{n} \sum_{t=1}^{n} -\log P(x_t \mid x_{<t}),$$

  onde:

  - n é comprimento da sequência;
  - P é dado pelo modelo de linguagem;
  - x_t é o *token* observado no t-ésimo passo.

- Essa medida torna comparáveis textos de comprimentos diferentes.

Avaliação de qualidade de modelos de linguagem

- Em processamento de linguagem natural (PLN), é comum usar a *perplexity*:

$$\begin{aligned}\text{perplexity} &= \exp(\text{entropia cruzada média}) \\ &= \exp\left(-\frac{1}{n}\sum_{t=1}^{n}\log P(x_t \mid x_{<t})\right).\end{aligned}$$

- Interpretação:
  - é o recíproco da média geométrica das probabilidades;
  - pode ser vista como o "número efetivo de opções" que o modelo considera em cada passo t.

Avaliação de qualidade de modelos de linguagem

- Casos extremos da *perplexity*
  - Melhor caso:
    - o modelo atribui probabilidade 1 ao *token* alvo;
    - perplexity $= 1$.
  - Pior caso:
    - o modelo atribui probabilidade 0 ao *token* alvo;
    - perplexity $= +\infty$.
  - Caso base:
    - o modelo atribui uma distribuição uniforme sobre todos os *tokens*;
    - perplexity $= |V|$, onde $|V|$ é o tamanho do vocabulário;
    - esse é um limiar que qualquer modelo útil deve **superar**: qualquer modelo razoável de linguagem precisa ter *perplexity* menor que $|V|$, ou seja, ser mais "informado" do que uma escolha puramente uniforme.

Resumo das RNNs

- Problemas das RNNs clássicas:
  - Desvanecimento de gradientes [*vanishing gradients*]
  - Explosão de gradientes [*exploding gradients*]
  - Processamento sequencial (lento)
  - Estrangulamento da capacidade / gargalo [*bottleneck problem*]
- Arquiteturas LSTMs e GRUs mitigam problemas que ocorrem RNNs clássicas:
  - Melhoram a modelagem de dependências de longo prazo (porém ainda de forma limitada).
  - Endereçam o problema do desvanecimento/explosão de gradientes.
  - Porém são mais lentas para treinar e não permitem processamento paralelo.
- Por essas razões, modelos baseados em redes recorrentes foram largamente suplantados por modelos baseados em **transformadores** [*transformers*].

Referências

- A. Géron, *Hands-On Machine Learning with Scikit-Learn: Concepts*, Tools and Techniques to Build Intelligent Systems, Keras & tensorFlow, Third Edition, O'Reilly, 2023. [capítulos 15 e 16]
- A. Zhang et al., *Dive into Deep Learning*, Cambridge University Press, 2023. [capítulos 9 e 10]
  - disponível em: `https://d2l.ai/`

Slides construídos com base nos livros supracitados.